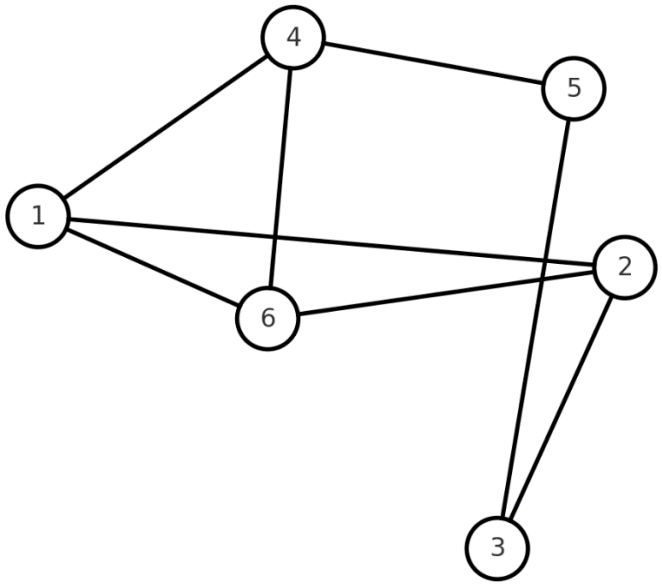
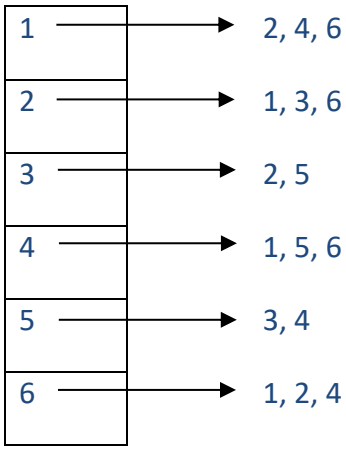
RUNI – Algorithms – solution to Homework 1

Problem 1:

An undirected graph G is given in the figure.



a. Draw the representation of G by adjacency lists. What is the size of the data structure if it takes 24 bits to store a header node (including a pointer to its adjacency list), and 32 bits to store a cell in a list? (use the actual number of vertices and edges in G to give your answer).



Headers: one per vertex → $6 \times 24 = 144$ bits.

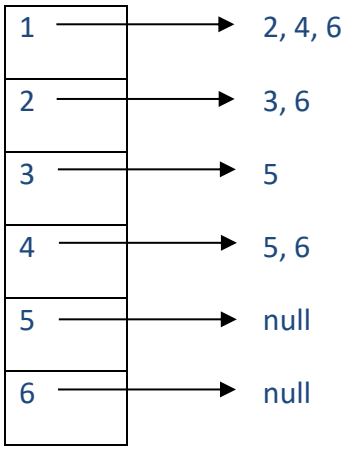
List cells: one per **undirected incidence** → $2 \mid E \mid = 16$ cells, each 32 bits → $16 \times 32 = 512$ bits.

Total: $144 + 512 = \mathbf{656}$ bits.

b. Convert G into a directed graph G': Let M be the adjacency matrix of G, then the adjacency matrix M' of G' is defined as follows: M'[i,j]=1 if and only if M[i,j]=1 and i<j.

i. Does G' contain a directed cycle?

ii. Repeat part ***a*** of the problem for G'.



No Cycle. All edges are from lower to higher label, so G' is acyclic.

Headers: one per vertex → $6 \times 24 = 144$ bits.

List cells: one per directed edge → $|E| = 8$ cells, $8 \times 32 = 256$ bits.

Total: $144 + 256 = \mathbf{400}$ bits.

c. Let G_1 and G_2 be two DAGs such that $V_1 = V_2 = \{a, b, c, d\}$. It is known that both graphs have the same unique (just one) topological sort. Prove or give a counter example: $E_1 \subseteq E_2$ or $E_2 \subseteq E_1$

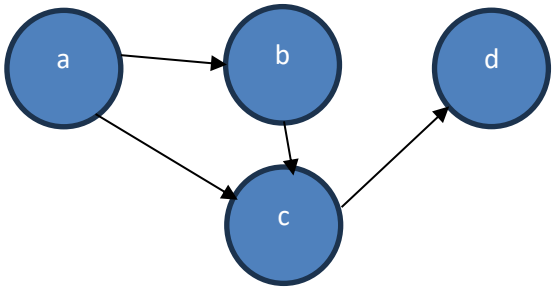
The statement is false

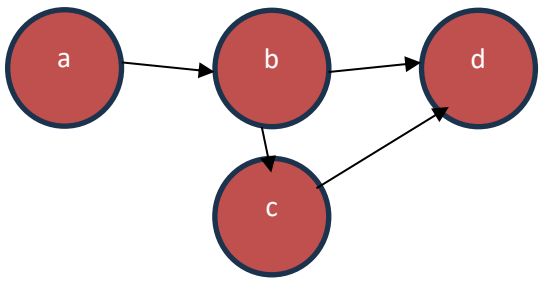
Counter example:

E1= (a,b),(a,c),(b,c),(c,d)

E2= (a,b),(b,c),(b,d),(c,d)

The unique topological sort is a,b,c,d





Problem 2:

In class we assumed that the topological sort algorithm is applied using a queue – the 'ready nodes' (those having in-deg=0) are enqueued into and dequeued from a queue. In fact, it is possible to use different data structures to keep the ready nodes, in particular, a stack. Here is the algorithm when applied with a stack:

Toposort using a stack:

1. Store each vertex's In-Degree in an array D
2. Initialize a stack with all "in-degree=0" vertices (push vertices with lower index first)
3. While the stack is not empty

   (a) pop and output a vertex

   (b) Reduce In-Degree of all vertices adjacent to it by 1

   (c) push to the stack any of these vertices whose In-Degree became zero (push vertices with lower index first)

4. If all vertices are output then success, otherwise there is a cycle.

Consider the following adjacency matrix M, which represents a directed graph G. $M(i,j) = 1$ if and only if there is an edge from i to j. Empty entries in the figure correspond to $M(i,j) = 0$.

	0	1	2	3	4	5	6
0							1
1				1	1		
2	1						
3							
4				1			
5		1			1		
6						1	

A. Specify the topological sort of G achieved by implementing the topological sort algorithm using a **stack**. In case of a tie, lower index is favored.
B. Analyze the time complexity of this implementation (the analysis should be general, as a function of $|V|$ and $|E|$ and not specific to the given graph). Specify the graph implementation you assume in your analysis (adjacency matrix or lists).

C. Assume the edge (1,2) is added to the graph. List all topological sorts that the resulting graph has or specify a cycle.

A. Topological sort achieved by the algorithm while implementing with a stack is: [2,0,6,5,4,1,3]

We now assume the graph is given in input using the array and adjacency lists representation.

Note that most of the details of the analysis stay same as the analysis shown in class for the implementation using a queue. Specifically, calculating the initial in-degree of all nodes is the same and is done in $O(|V| + |E|)$ time. The only difference is in the maintenance of the next nodes candidates to be output. Instead of using the queue methods of insertion and removal (enqueue, dequeue), we use the stack methods (pop, push) – but all these methods are performed in $O(1)$ time, so the time complexity is not influenced by replacing enqueue with push, or by replacing dequeue with pop.

B. Therefore, even this part still has the same complexity analysis as in class. All in all, the running time is $O(|V| + |E|)$, as shown in class.

(If you assume adjacency matrix representation, the time complexity is still the same as with queue implementation - $O(|V|^2)$)

C. Not a DAG, cycle [1,2,0,6,5,1]

Problem 3:

Definition: Let $G = (V, E)$ be a graph. Let $u, v \in V$ be two vertices in the graph. We say that v is reachable from u, if there is a path in G which starts with the vertex u and ends with v.

Definition: A directed acyclic graph $G = (V, E)$ is said to be semi-connected if for any two vertices v and u, v is reachable from u, or u is reachable from v.

Give a linear-time algorithm to determine whether a DAG G is semi-connected. Supply an algorithm, then prove that your algorithm is correct, and then analyze its time complexity

Algorithm:

1. Find a topological sort of G: $v_1, v_2, \dots, v_n$
2. For every $1 \leq i < n$: check if there is an edge from v_i to v_{i+1}
3. If all such edges are in the graph return True. If such an edge is missing return False.

Correctness:

Observation: According to the topological sort definition for every two nodes v_i, v_j : v_j may be reachable from v_i only if $j > i$. That's because all edges of the graph go from left (lower index) to right (higher index) in this sequence.

For G to be semi-connected, a necessary condition is that there will be a path between v_i and v_{i+1} (or the other way around) for every $1 \leq i < n$. According to the observation above, this only occurs if there is a direct edge from v_i to v_{i+1}. So, step (2) checks this necessary condition.

Also note this condition is sufficient: if such an edge between any two consecutive nodes exists, then for every two nodes in the graph v_i, v_j there is a path from one to the other. Explanation: suppose $j > i$, then if the check in step (2) is fulfilled, the sequence $v_i, v_{i+1}, \dots, v_j$ is a valid path in G from v_i to v_j.

Time complexity:

Step 1 is achievable in $O(|V| + |E|)$ time, as shown in class.

Step 2 is done by iterating all the nodes in the order of the topological sort, and for each we iterate once all its outgoing edges (the list of its neighbors). This is also achievable in $O(|V| + |E|)$ time.

Total running time: $O(|V| + |E|)$.

Problem 4:

Let $G = (V, E)$ be a directed graph. Let $G' = (V, E')$ be the graph obtained by reversing the edge directions in G. Formally, $E' = \{(v, u) \mid (u, v) \in E\}$.

Assume that G has exactly 650 different topological sorts. What can be said on the number of topological sorts of G'? Although the instruction is vague, you should supply the tightest possible bounds on the actual answer. Prove the statements you make.

Statement: $S = v_1, v_2, \ldots . v_n$ is a topological sort of G if and only if the reverse sequence $S^{-1} = v_n, v_{n-1}, \ldots, v_1$ is a topological sort of G'.

Notice that the statement reveals the number of topological sorts of G and the number of topological sorts of G' are equal. So, it must be 650 as well.

Proof of statement:

Assume $S = v_1, v_2, \ldots, v_n$ is a topological sort of G.

By definition, if we "place" all the nodes in the order of S from left to right, all edges in E are directed from left to right.

Since the edges of the graph G' are the same edges reversed, if we "place" all nodes in the order of S from left to right, all edges in E' are directed from right to left.

By reversing the order of this sequence, we get the sequence S^{-1}, in which all edges of G' are directed from left to right. This means that S^{-1} is indeed a valid topological sort of G'.

By the symmetry nature of this statement the proof of the other direction is the same.

Problem 5:

Let G = (V, E) be a simple undirected graph. The **complement graph** of G is defined as follows: $G^c = (V, E^c)$, where $(u,v) \in E^c$ iff $(u,v) \notin E$. That is, the set of nodes of G^c is the same as that of G, and there is in an edge in G^c connecting any two nodes that do not have an edge connecting them in G.

a. (6 pts.) Describe an undirected graph G with n=4 nodes, such that both G and G^c are connected. Draw both G and G^c.

G: The path A-B-D-C

G^c: The path: B-C-A-D

b. (12 pts.) Prove: For every simple graph with n>2 nodes, if G^c is not connected, then G is connected.

Let u,v be two vertices in G. We show that G includes a path connecting u and v.

If $(u,v) \in E$, then u and v are connected, if $(u,v) \notin E$ then u and v are in the same connected component of G^c. Let a be some vertex in a different connected component of G^c. By definition of G^c, the set E includes the edges (u,a) and (v,a), implying that u and v are connected in G by the path u-a-v.